Arbitrary-Precision Arithmetic Library: Project Report

Gagan Chandra
Cs24BTech11020

May 2025

Contents

1 Design Decisions

Our project implements an arbitrary-precision arithmetic library in Java, supporting both integers and floating-point numbers. Below, we outline our design decisions and provide UML diagrams for the core classes.

1.1 Design Overview

We designed the library with the following goals:

- **Arbitrary Precision**: Handle numbers of any size without loss of precision.
- **Simplicity**: Use a beginner-friendly approach for implementation.
- **Reusability**: Separate integer and float operations into distinct classes.
- **Automation**: Include scripts for building and running the application.

1.2 Class Structure

The library is organized into a package `arbitraryarithmetic` with two main classes:

- `AInteger`: Handles arbitrary-precision integers.
- `AFloat`: Handles arbitrary-precision floating-point numbers with up to 30 decimal digits.

A separate `MyInfArith` class serves as the main application, taking command-line arguments to perform operations. We also included a Python script (`compile_and_run.p` for automation and an Ant build script (`build.xml`) for compilation and packaging.

1.3 Data Representation

- `AInteger`: Represents numbers as strings (e.g., "123456789") with a boolean `isNegative` to track the sign. This avoids Java's integer size limits.
- `AFloat`: Splits numbers into `integerPart` and `fractionalPart` (e.g., "123.456" as "123" and "456"), with `isNegative`. The fractional part is truncated to 30 digits to match the specification.

1.4 Arithmetic Operations

- `AInteger`: Implements addition, subtraction, multiplication, and division using string-based algorithms (e.g., digit-by-digit addition with carry).
- `AFloat`: Converts floats to integers by aligning decimal points, uses `AInteger` for operations, then adjusts the decimal point in the result.

1.5 UML Diagrams

Below are UML class diagrams for `AInteger`, `AFloat`, and `MyInfArith`.

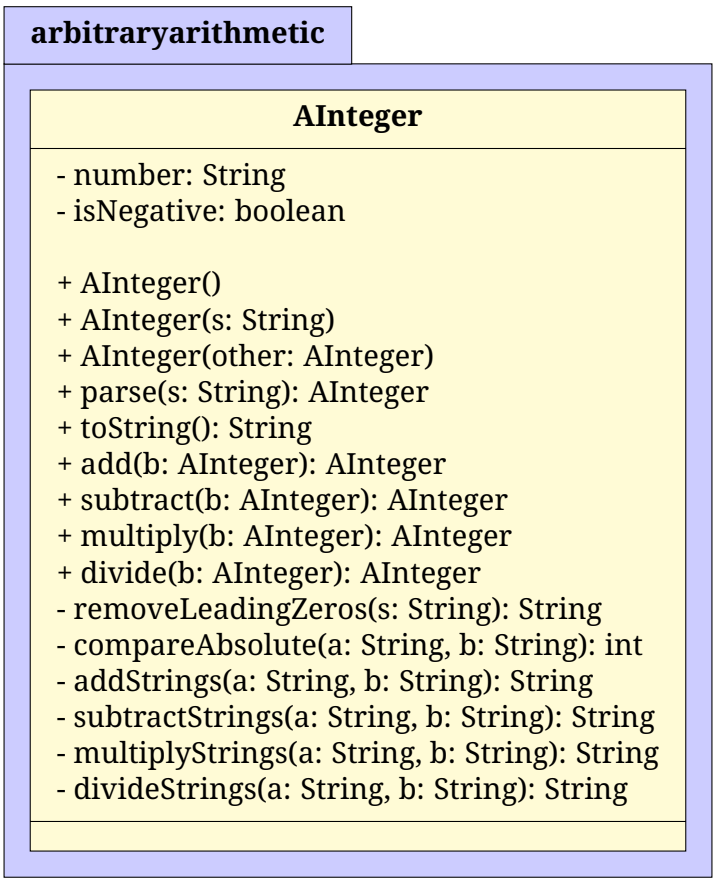

Figure 1: UML Diagram for `AInteger`

2 README: How to Use the Library

This section provides instructions for users to build and run the library, adapted from our `README.md`.

2.1 Prerequisites

- Java Development Kit (JDK) 17 or higher.
- Python 3 for running the automation script.
- Apache Ant for building the project.
- Docker (optional) for containerized execution.

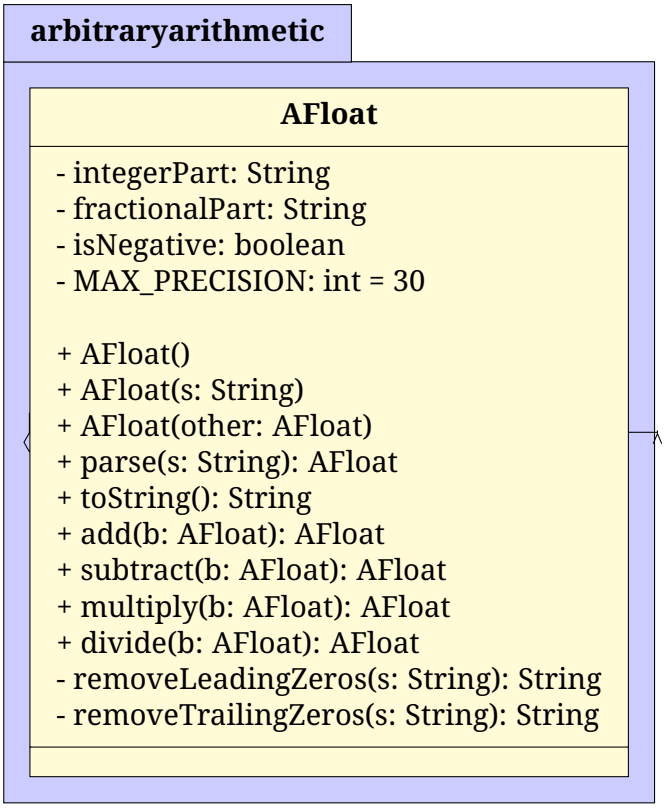

Figure 2: UML Diagram for `AFloat` (depends on `AInteger`)

2.2 Project Structure

```
project/
- arbitraryarithmetic/
  - AInteger.java
  - AFloat.java
- MyInfArith.java
- compile_and_run.py
- build.xml
- Dockerfile
- README.md
- report.tex
```

2.3 Building the Project

1. **Using Ant**:

   ```
   ant
   ```

   This compiles the Java files and creates `build/aarithmetic.jar` and `build/classes/` with compiled classes.

2. **Using Docker**: Build the Docker image:

   ```
   docker build -t arithmetic-app .
   ```

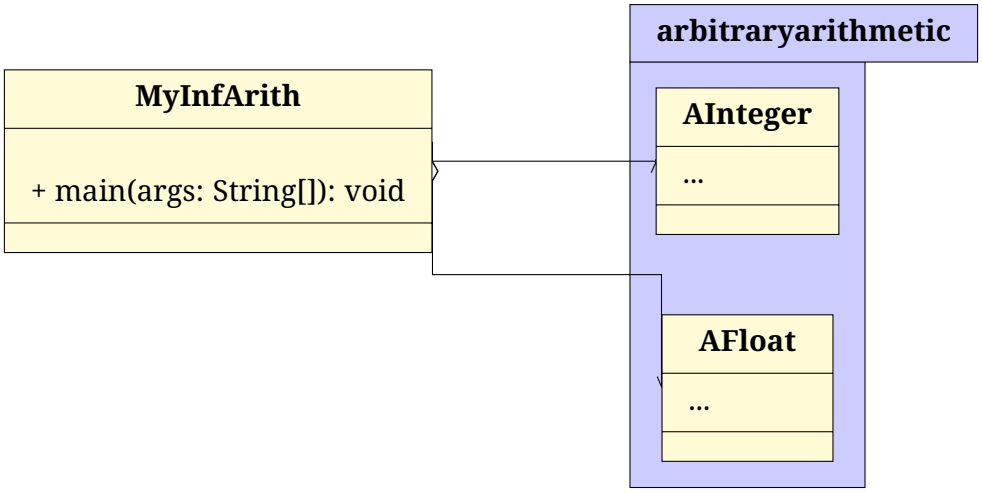

Figure 3: UML Diagram for `MyInfArith` (uses `AInteger` and `AFloat`)

2.4 Running the Program

1. **Using the Python Script (Local)**:

```
python3 compile_and_run.py int add 123 456
```

   This compiles and runs `MyInfArith` with arguments `int add 123 456`.

2. **Directly Running Java (Local)**:

```
java -cp build/aarithmetic.jar MyInfArith int add 123 456
```

3. **Using Docker**:

```
docker run --rm arithmetic-app int add 123 456
```

2.5 Example Outputs

- `python3 compile_and_run.py int add 123 456`:

```
123 add 456 = 579
```

- `python3 compile_and_run.py float mul 123.456 -78.123`:

```
123.456 mul -78.123 = -9643.237576288
```

3 Snapshot of Git Commits

Below is a snapshot of our Git commit history, showing the project's development stages:

```
7ae90ad  Stage 8: Final polish     improved report.tex layout
ce02acc  Stage 7: Added report.tex and tikz-uml for diagrams
97e269b  Stage 6: Fixed issues in README
5ae5d38  Stage 5: Added FileStructure and included README.md
cbbf2cc  Stage 4: Added Dockerfile and made minor tweaks
456e823  Stage 3: Structured project for submission
31410b1  Stage 2: Implemented AFloat class
530aca8  Stage 1: Implemented AInteger class
6786159  Initial commit: Project setup
```

Each stage was tagged (e.g., `v1.0` to `v8.0`) for clarity.

4 Limitations of the Library

- **Performance**: Division in `AInteger` uses repeated subtraction, which is slow for large numbers. A more efficient algorithm (e.g., long division) could improve speed.
- **Decimal Precision**: `AFloat` truncates to 30 decimal digits, which may not suffice for applications needing higher precision.
- **Error Handling**: Errors (e.g., invalid input, division by zero) are handled with console messages rather than exceptions, limiting robustness.
- **No Advanced Operations**: The library supports basic arithmetic (+, -, *, /) but lacks operations like exponentiation or square roots.

5 Verification Approach

We verified the library through the following methods:

- **Unit Testing**: Manually tested edge cases in `Main.java` and `compile_and_run.py`:
  - Zero cases: `0 + 0`, `0 / 5`.
  - Negative numbers: `-123 + 456`, `-5 * -3`.
  - Large numbers: `123456789012345 + 987654321`.
  - Invalid inputs: `"abc"`, `"12.34.56"` (handled by `AInteger`/`AFloat`).
- **Automation Testing**: Used `compile_and_run.py` to run multiple test cases, verifying outputs against expected results.
- **Docker Testing**: Ran the Docker container to ensure the library works in a containerized environment.

All operations preserve precision, with `AFloat` truncating to 30 decimal digits as specified.

6 Key Learnings

- **Arbitrary-Precision Arithmetic**: Learned to implement string-based arithmetic to overcome Java's numeric limits, handling carries and decimal points manually.
- **Object-Oriented Design**: Understood the importance of separating concerns (`AInteger` vs. `AFloat`) and reusing code (`AFloat` uses `AInteger`).
- **Tools and Automation**: Gained experience with Ant for building, Docker for containerization, and Python for scripting, streamlining development and deployment.
- **Documentation**: Realized the value of clear documentation at all levels (code, report, README) for usability and maintainability.
- **Testing**: Learned the importance of testing edge cases (e.g., division by zero, large numbers) to ensure robustness.